

R2D2: Robotized Reconfigurable Network for Disaggregated Datacenters

Linus Y. Wong^{*†}, Zhiyao Tang^{*†}, Justin R. Yu[†], Zhilei Zheng^{§‡}, Jonathan M. Smith[†], André DeHon^{†‡}, Jing Li^{†‡}

[†]Department of Electrical and Systems Engineering, University of Pennsylvania, Philadelphia, Pennsylvania, USA

[‡]Department of Computer and Information Science, University of Pennsylvania, Philadelphia, Pennsylvania, USA

Emails: {lywong, zhiyao, justinyu, zhileiz}@seas.upenn.edu, jms@cis.upenn.edu, {andre, janeli}@seas.upenn.edu

Abstract—Disaggregated datacenters present unique network challenges. Current network hardware serving disaggregation traffic is spatially overprovisioned with statically configured all-to-all connectivity and temporally overprovisioned with static, per-packet switching costs, regardless of actual network traffic patterns. This leads to significant underutilization and reduces the cost benefits of resource disaggregation.

To address this, we propose R2D2, a robot-enabled reconfigurable datacenter network architecture. Through software-hardware co-design, R2D2 exploits the spatial and temporal locality of network traffic to achieve efficient network specialization. In particular, for the first time, R2D2 leverages mature commodity robotics to reconfigure the network and establish direct, single-hop physical connections between disaggregated resources on demand, eliminating the spatial overprovisioning of all-to-all connections. Furthermore, R2D2 *dynamically* reconfigures the network on demand to adapt to changing traffic patterns, thereby removing the need for continuous per-packet switching; hence, avoiding temporal overprovisioning. Lastly, we introduce a co-optimized runtime system, co-designed with the robotized network architecture, to improve performance, resource-efficiency, and availability. We demonstrated the feasibility of R2D2 with comprehensive experimental evaluations and a robot prototype, using real-world applications and datacenter allocation traces. Our results show that R2D2 achieves up to 38% reduction in network costs and up to 84% savings in network energy consumption, while delivering comparable end-to-end application performance compared to state-of-the-art fat-tree and OCS networks.

Index Terms—Memory Disaggregation, Reconfigurable Networks, Datacenter, Cloud Computing.

I. INTRODUCTION

Datacenters are increasingly disaggregated, with hardware components traditionally collocated within the same physical server enclosure now being separated into independently managed resource pools [1], [3], [9], [12], [14], [17], [25], [26], [30], [31], [35], [39], [41], [42], [57], [60], [61], [68], [69], [74], [75]. Disaggregation aims to improve the utilization of disaggregated resources like CPU, memory, and storage, thereby reducing the operational costs.

However, this shift increases network costs, which diminish the anticipated overall cost savings from disaggregation [1]. In a disaggregated datacenter (DDC), traditional *local* inter-

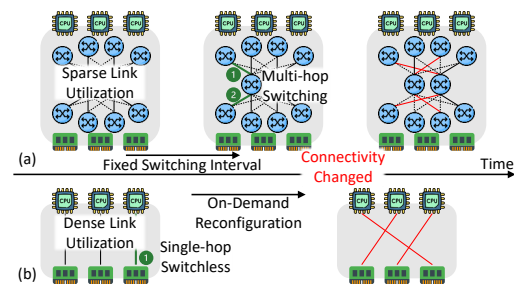


Fig. 1: Comparison of conventional network and R2D2.

connects including chip-level connections (e.g., processor or memory bus) and server-level connections (e.g., PCIe) are replaced by datacenter-scale or rack-scale *network* that connects separate pools of resources (compute, memory, storage) distributed across servers. This generates significant network traffic as system software now needs to access remote resources over an “external” network, instead of local interconnects within the server. Notably, *disaggregation traffic* — distinct from *application traffic* between distributed applications — is generated by system software for essential management and access to disaggregated resources, requiring ultra-low latency (sub-10 μ s) and high bandwidth (40–400 Gbps) [25]. To serve disaggregation traffic, DDCs commonly employ network designs similar to those used in non-disaggregated traditional datacenters, but provision a costly, fully connected, high-bandwidth, and RDMA-capable network with high-end network switches and network interface cards (NICs), dedicated to disaggregation and separate from application traffic. In these networks, cables are *statically* wired to provide high-bandwidth all-to-all connectivity, and switches are *statically* configured to perform *fixed*, *per-hop* packet switching, regardless of the network traffic.

Nevertheless, this *static*, *traffic-agnostic*, *one-size-fits-all* network design leads to significant overprovisioning in hardware, undermining disaggregation cost advantages. Our network traffic analysis identifies two types of over-provisioning: spatial and temporal. **Spatial overprovisioning** arises because current network hardware statically provisions *all-to-all* connections (cables) between servers, despite disaggregation traffic exhibiting >98% sparsity — communication

^{*}Both authors contributed equally to the paper.

[§]Work done while at University of Pennsylvania.

happens only between a small subset of compute and memory nodes allocated to the same task, leaving most links unused. **Temporal overprovisioning** stems from the current network hardware design aiming to maximize capacity for a worst-case, constantly changing traffic pattern at the finest-grained *per-packet* switching granularity (100s ns). However, disaggregation traffic forms stable, long-lived flows between compute-memory node pairs (minutes to hours), underutilizing fine-grained packet switching. Our study further show that common task placement policies, designed for performance and resource utilization in DDCs, exacerbates overprovisioning.

In a nutshell, the current DDC network hardware is spatially overprovisioned with all-to-all connectivity and temporally overprovisioned with per-packet switching rate, thereby diminishing the cost-saving benefits of resource disaggregation. To address this issue, we propose R2D2- **Robotized Reconfigurable Network for Disaggregated Data-** centers – a *switch-less*, connection-oriented network architecture using hardware/software co-design. R2D2 leverages robotic reconfiguration to establish direct physical connections between disaggregated resources (CPU, memory, storage) only as needed, eliminating the spatial overprovisioning of all-to-all connections. Furthermore, R2D2 *dynamically* reconfigure the network on demand to adapt to changing disaggregation traffic patterns (e.g., VM launch), thereby removing the need for continuous per-packet switching and avoiding temporal overprovisioning. This is in stark contrast to state-of-the-art (SOTA), switch-based, statically provisioned networks that rely on all-to-all, always-on switching and are agnostic to actual traffic patterns.

We note that R2D2 is inspired not only by the spatial sparsity and temporal stability of disaggregated traffic patterns, but also by recent advances in robotics enabling low-cost, high-precision automation via gantry robots widely used in 3D printing and semiconductor manufacturing. Specifically, R2D2 repurposes commodity gantry robots for network reconfiguration in DDCs. Gantry robots offer sub-10 μm repeatable precision over a large operation envelope at lower cost than articulated arms. Our feasibility analysis (§III-A) shows these properties fit the demands of disaggregation traffic: the large operation envelope leverages spatial sparsity to enable high radix, while the reconfiguration speed (10 s) suffices to serve the stable temporal changes (minutes to hours). Thus, gantry robots are well-suited to realize R2D2’s adaptive, runtime physical network reconfiguration.

The R2D2 system comprises two main components: (1) a hardware architecture built from an optical fiber fabric dynamically reconfigured by repurposed commodity gantry robots integrated into datacenter racks, enabling single-hop, low-latency, high-bandwidth connectivity between disaggregated resources by eliminating all-to-all wiring or per-packet switching; (2) a software runtime that co-optimizes physical topology and task placement by coordinating reconfiguration across datacenter system controller and embedded robotic controllers while actively engineering traffic patterns to promote sparse and long-lived connections. Each R2D2 robotic unit has high radix and can serve thousands of nodes, enabling

datacenter-wide connectivity with at most two levels of hierarchy. The software layer spans global resource management and robot control, with a hierarchical design decoupling high-level scheduling from low-level motion planning to ensure scalability and responsiveness. Reconfiguration costs are incorporated into placement decisions to maintain sparse and stable network topologies and minimize robotic movements. Details on the R2D2 hardware and software components are in §IV and §V.

The R2D2 architecture reduces cost and energy consumption by leveraging mature, commodity robotic platforms to dynamically (re-)configure physical links *on demand*, eliminating the spatial and temporal overprovisioning inherent in statically configured, all-to-all, packet-switching networks. By repurposing off-the-shelf components and provisioning only the necessary connectivity and reconfiguration speed, R2D2 achieves lower costs than modern 400GbE switch-based architectures. This cost advantage scales with system size: the high-radix of R2D2 units enables datacenter scale connectivity with at most two levels of hierarchy. The required robotic units and fiber links grow only linearly with system scale, maintaining scalability and cost-efficiency. In contrast, traditional switch-based networks require deeper hierarchies with more nodes, leading to superlinear growth in cost per node. In energy use, R2D2’s direct connection approach decouples maintenance of active data paths from topology updates, consuming no power for sustained dataflows and only during infrequent reconfigurations. Even during reconfiguration, the transient peak power ($\sim 120\text{W}$) is below the static power consumption of a lightly loaded traditional switch ($\sim 400\text{W}$). Crucially, R2D2 achieves these cost and energy advantages without sacrificing performance. Unlike conventional switched fabrics, it establishes direct, point-to-point physical links, eliminating per-hop latency and shared bandwidth bottlenecks, allowing endpoints to communicate at the full line rate.

We evaluate R2D2 in terms of cost, power, and performance on real-world benchmarks, using network traces from disaggregated applications [25] and allocation traces from production datacenters [32]. Our prototype, built with a commodity 3D printer and fiber patch panels/cables, measures the cost, power, latency, and throughput of robotic reconfiguration. Compared to a SOTA fat-tree and OCS baselines, R2D2 reduces hardware cost by up to 38% and energy consumption by up to 84%. R2D2 also improves flow completion time by up to 43.3% and throughput by up to 70.2% over fat-tree. At the application level, R2D2 can achieve performance comparable to that of fat-tree and OCS networks.

We summarize our main contributions as follows:

- We present the first in-depth characterization of traffic in disaggregated datacenters, identifying and quantifying the significant spatial and temporal locality that leads to network overprovisioning. Guided by our discovery, we further demonstrate the feasibility of using commodity gantry robotics to physically reconfigure network links to meet these traffic demands.
- We design and present *R2D2*, a novel robotized reconfigurable network architecture that, for the first time,

leverages commodity gantry robots to dynamically reconfigure direct physical links between compute and memory nodes. By physically adapting the network topology to traffic patterns, *R2D2* exploits the identified spatial and temporal locality to directly address the cost and energy inefficiencies of static, overprovisioned networks.

- We develop a co-designed runtime system that jointly manages both task scheduling and robotic network reconfiguration. Our system features a hierarchical design for scalability and a novel scheduling algorithm that, by incorporating the physical reconfiguration cost, promotes sparse and stable network connections.
- We present a comprehensive evaluation using real-world applications through a combination of prototyping and system simulation. The results demonstrate that *R2D2* reduces network hardware's cost by up to 38% and energy consumption by up to 84%, while maintaining comparable end-to-end performance with SOTA fat-tree and OCS baselines.

II. BACKGROUND AND MOTIVATION

A. Network Cost for Resource Disaggregation

The network is a critical component in disaggregated datacenter architectures with significant cost and energy overheads [1]. It serves two types of traffic: *disaggregation traffic*, which enables communication between disaggregated resources such as compute, memory, and storage; and *application traffic*, which supports communication among applications running on the servers. Disaggregation traffic is essential to system functionality for resource access and system-level management, thus demands high bandwidth (40–400Gbps) and ultra-low latency (sub-10 μ s). To meet these stringent requirements, prior work on disaggregated systems [3], [25], [30], [57], [61], including commercial datacenters [9], typically adopts a costly, fully connected, RDMA-capable fat-tree network dedicated exclusively to disaggregation traffic and isolated from application traffic, significantly increasing both network cost and energy consumption [1].

To quantify these overheads, we evaluate two fully connected RDMA networks with 512 and 2048 nodes, as detailed in Table I. Both use SOTA 400GbE switches, NICs, and servers with NICs directly attached to CPUs [6]. The network accounts for 23.1% of system cost and 21.3% of power at 512 nodes, and 23.0% and 25.7%, respectively, at 2048 nodes, underscoring the substantial cost and energy overheads of disaggregated networking.

B. Gantry Robots

Gantry robots are widely used in cost-sensitive automation because they offer sub-10 μ m repeatable positioning accuracy at a cost one to two orders of magnitude lower than articulated arms of comparable reach. A typical Cartesian gantry robot consists of three primary subsystems: an End-effector (e.g., gripper, nozzle, probe) mounted on a moving carriage; Orthogonal guide rails that constrain motion to three perpendicular

axes; and Linear drive modules that actuate the end-effector within its operational envelope (Figure 2).

The performance and cost advantages of gantry robots stem from their orthogonal structure, which kinematically decouples the axes. This design natively supports Cartesian motion, avoiding costly inverse kinematics, and has high positioning accuracy, since errors are bounded to a single axis rather than accumulating across joints. As shown in Table II, commodity hardware with simple controllers can achieve sub-10 μ m accuracy over meter-scale envelopes. This combination of low cost and high precision makes gantries ideal for tasks such as 3D printing, PCB assembly, and semiconductor fabrication.

C. Disaggregation Traffic Analysis

We analyze network traffic to characterize disaggregation traffic patterns and identify opportunities for network specialization. Using VM traces from production datacenters [32], we examine **spatial sparsity** and **temporal stability** of disaggregation traffic. Detailed setup can be found in §VI-A.

We evaluate three representative resource placement policies from prior works [3], [17], [30], [31], [57], [61]: 1) *Random-fit*: Randomly assigns compute and memory nodes, prioritizing scheduling speed [3] or load balancing [17], resulting in a worst case for sparsity and stability. 2) *Best-fit*: Assigns compute and memory nodes to minimize fragmentation [30] while treating all compute-memory pairs uniformly. 3) *Topology-aware*: Prioritize assignment to compute-memory pairs with existing affinity [31], [57], [61], improving sparsity and stability while maintaining utilization.

Spatial Sparsity: We construct heatmaps of steady-state compute-memory traffic (Fig. 3(a)-(b)), following TVMPP [51] and ECHO [15]. At 512 nodes, 98.8%, 99.1%, and 99.5% of links carry no traffic under random-fit, best-fit, and topology-aware placement, respectively, equivalent to each compute node using only 6, 5, and 3 of 511 possible memory links. Sparsity increases in the 2048-node system (Fig. 3(b)) to 99.7%, 99.8%, and 99.8%, or roughly 6, 4, and 4 active links per node, indicating that sparsity persists across scales.

Temporal Stability: We measure the fraction of stable compute-memory connections over time (Fig. 3(c)-(d)), following Infiniswap [30] and Sinbad [13]. At 512 nodes, temporal stability remains above 0.99 across 1000-3000 s interval for all policies, and exceeds 0.997 for topology-aware. At 2048 nodes, temporal stability is above 0.996 across all intervals. This stability arises from long VM lifetimes [32] and, for topology-aware, explicit compute-memory node bindings.

The above **spatial sparsity** and **temporal stability** contrast starkly with the fully connected and ns-to- μ s per-hop packet switching design of current disaggregation networks, leading to **spatial** and **temporal overprovisioning**.

Spatial Overprovisioning: Current disaggregation network designs provide high-bandwidth all-to-all connectivity. However, our analysis demonstrates that disaggregation traffic, especially under topology-aware placement, exhibits up to 99.8% spatial sparsity. This signifies a vast majority of potential links carry no traffic, rendering allocated connectivity highly

Components	512-node Setup			2048-node Setup		
	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty
FS 32x400Gb Switch (N8610-32D) [23]	18.8K	1000	48	—	—	—
FS 64x400Gb Switch (N9510-64D) [24]	—	—	—	37.3K	2524	96
Total switch power and cost	902K	48.0K	—	3581K	242K	—
Total nodes supported	—	—	512	—	—	2048
Switching per node	1762	93.8	—	1748	118	—
SuperMicro 222HA-TN Dual Socket [67]	26366	1100	256	26366	1100	1024
Intel Xeon 6952P 96 Core CPU	Included	Included	2	Included	Included	2
64GB DDR5-6400 DRAM	Included	Included	24	Included	Included	24
Broadcom P1400GD 400Gb NIC [8]	2198	75	2	2198	75	2
Total per node	17143	550	—	17129	550	—
Switching per node (as %)	3960 (23.1%)	117 (21.3%)	—	3946 (23.0%)	141 (25.7%)	—

TABLE I: Cost and power breakdown for 512-node and 2048-node fully-connected network configurations.

underutilized. For instance, in a system with 2048 nodes, 99.8% sparsity means each machine actively communicates over approximately 4 links out of 2047 possible connections. This spatial overprovisioning implies future disaggregation networks could benefit from designs exploiting orders of magnitude sparser connectivity, leading to substantial reductions in hardware cost and power consumption.

Temporal Overprovisioning: Current disaggregation networks are engineered for extremely low-latency, dynamic ns-to- μ s per-hop packet switching. This design is optimized for highly bursty and unpredictable general-purpose datacenter traffic. In stark contrast, our findings reveal that disaggregation traffic patterns are remarkably stable over minutes to hours (1000s of seconds). The architectural choice of fine-grained, per-packet routing for traffic that changes on such a significantly slower timescale represents severe temporal overprovisioning. This profound mismatch implies that disaggregation networks could operate at orders of magnitude coarser time granularities (e.g. minute-to-hour basis), substantially reducing control overhead and offering significant opportunities for network cost and power savings.

III. R2D2 OVERVIEW

A. Feasibility of Commodity Robot Reconfiguration

In §II-C, we demonstrated that the spatial sparsity and temporal stability of disaggregation traffic mismatch the design assumptions of current disaggregated datacenter networks leading to significant overprovisioning. Consequently, we ask: *Can we leverage commodity robots to reconfigure the physical network topology to exploit the spatial and temporal locality of disaggregation traffic, thus eliminating the need for complex, overprovisioned network switching?* This subsection evaluates the feasibility of commodity robotics in meeting the spatial connectivity and temporal reconfiguration demands of disaggregation traffic.

a) Spatial Connectivity: As shown in our spatial traffic analysis (§II-C), disaggregated traffic is sparse but still requires a modest degree of concurrent connectivity per node. Specifically, with a topology-aware placement strategy, each compute node communicates concurrently with approximately 3-4 memory nodes in 512-node and 2048-node clusters, and vice versa. This bidirectional fanout of 4 represents the minimum degree of concurrent links needed to support flexible

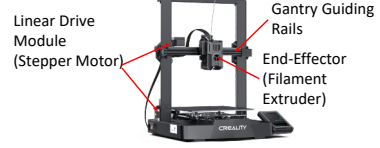


Fig. 2: A commodity 3D Printer (Gantry Robot)

Item	Cost	Power	Speed	Precision	Build Size
Creality Ender-3 V3 KE [46]	\$279	350W	0.6m/s	0.1mm	0.22m × 0.22m × 0.24m
Prusa MK4S [49]	\$729	120W	0.6m/s	0.1mm	0.25m × 0.21m × 0.22m
Elegoo OrangeStorm Giga [47]	\$2499	1524W	0.3m/s	0.1mm	0.8m × 0.8m × 1.0m

TABLE II: Cost, power, speed, precision, and size of different commodity 3D printers.

allocation and reduce resource stranding. With random and best-fit placement, the required number of active connections slightly increases to at most 6—still significantly below the theoretical worst case of 511 and 2047 connections per node in fully connected topologies for 512-node and 2048-node clusters, respectively.

Hence, the spatial sparsity of disaggregation traffic makes it possible to reconfigure the physical network topology while staying within both the rack’s spatial budget and the reach-and-accuracy envelope of commodity gantry robots.

Concretely, an 4-way fan-out over a 512-compute/512-memory pod requires $512 \times 4 = 2048$ unique compute-to-memory links and thus 4096 connection points to be managed by the reconfiguring robots. To give a scale of size, existing fiber panels used in datacenter today can pack 96 fiber connectors per 4.5cm rack units. Utilizing both front and back of the rack unit, we can maximally pack $96 \times 2 = 192$ connectors per rack unit. As a result, the rack space required to accommodate the compute-memory pods of a total of 1024 machines is $\frac{4096}{192} \times 4.5 \text{ cm} \approx 96 \text{ cm} \approx 21.3 \text{ U}$ occupying about half of a 48U rack while leaving 2U buffer space for power, controllers, and the robot’s home dock. For a 1024-compute/1024-memory pod, the requirement is $\approx 42.6 \text{ U}$.

Meanwhile, largest commodity 3-D printer offers an 80 cm by 80 cm planar reach with 100 cm Z-clearance, comfortably covering the 48.3 cm (19in.)-wide rack face. Because the port count and the corresponding rack space used scale linearly with fan-out and node count, doubling the cluster size or modestly increasing fan-out still fits within one (or at most two) racks. Consequently, the small active-link budget afforded by spatial sparsity keeps the R2D2 fabric within the service envelope of a single, low-cost commodity 3-D printer.

b) Temporal Reconfiguration: Disaggregation traffic exhibits temporal stability over timescales ranging from minutes to hours (i.e., thousands of seconds), as characterized in §II-C. This property offers a compelling opportunity to exploit commodity robotic systems for physical reconfiguration. In contrast to traditional networks that operate at nanosecond timescales, robotic reconfiguration on the order of tens of seconds is more than sufficient.

In our running example, the 512-compute/512-memory pod occupies just 21.3 U – about 100 cm of vertical space – across the rack’s 48 cm width. The longest possible move, between

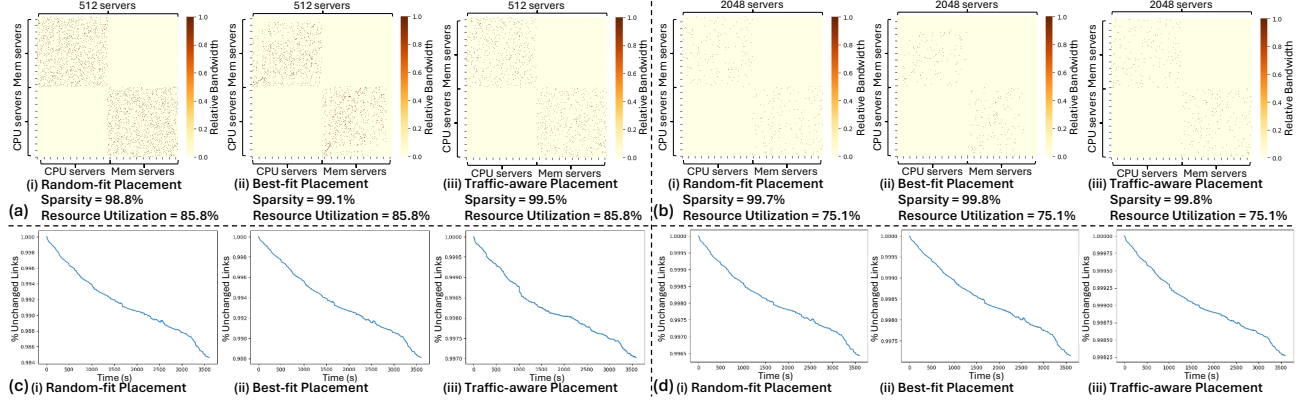


Fig. 3: (a) Disaggregation traffic heatmap for three different placement strategies at 512 nodes: (i) Random-fit, (ii) Best-fit, (iii) topology-aware. (b) Disaggregation traffic heatmap for three different placement strategies at 2048 nodes: (i) Random-fit, (ii) Best-fit, (iii) topology-aware. (c) Percentage of connection that remains unchanged after time at 512 nodes for (i) Random-fit, (ii) Best-fit, (iii) topology-aware. (d) Percentage of connection that remains unchanged after time at 2048 nodes for (i) Random-fit, (ii) Best-fit, (iii) topology-aware.

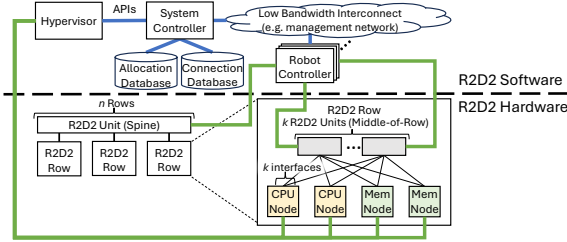


Fig. 4: Overview of R2D2, a hardware-software co-designed system for disaggregation network.

diagonal corners, spans $\sqrt{(100\text{ cm})^2 + (48\text{ cm})^2} \approx 111\text{ cm}$. At a conservative 300 mm s^{-1} (readily achieved by commodity XYZ gantries; see §II), the worst-case traversal time is $111\text{ cm}/300\text{ mm s}^{-1} \approx 3.7\text{ s}$; for a 1024-compute/1024-memory pod, the corresponding worst case is $\approx 7.3\text{ s}$.

Including overheads for motion control, alignment precision, cable insertion and removal, and cable management, we conservatively estimate that a full reconfiguration can be completed in under 15 seconds. This sub-minute operation time is orders of magnitude faster than the timescale of disaggregation traffic dynamics. Thus, the temporal slack is sufficiently large to make robotic physical network reconfiguration not only feasible but also practical for real-world datacenter operations.

B. R2D2 System Overview

Having established the feasibility of using commodity robot reconfiguration to meet the demands of disaggregation traffic in §III-A, we now present the system overview of *R2D2* (**R**obotic **R**econfigurable **D**isaggregated **D**atacenter). *R2D2* is a hardware/software co-designed system that exploits the spatial sparsity and temporal stability of disaggregation traffic and harnesses the explosive growth of commodity robots to create a cost-effective, power-efficient, and low-latency network fabric. The system is built upon two tightly integrated layers: a hardware layer of *R2D2* robots and a software

runtime that actively co-optimizes task placements and the physical topology reconfiguration in response to dynamically changing disaggregation traffic.

The hardware layer (§IV) consists of a physical network fabric built from direct fiber optic connections, reconfigured on demand by robots. This direct, single-hop connectivity eliminates many layers of hierarchical switching, delivering the low-latency, high-bandwidth communication essential for memory disaggregation. By repurposing commodity Cartesian robots and leveraging the speed, precision, cost-effectiveness, and power efficiency of 3D-printer technology, *R2D2* enables low-cost, energy-efficient network reconfiguration. A single robot can manage a meter-scale reconfiguration patch panel serving up to 4608 nodes, allowing the architecture to scale to datacenter-scale with a two-level hierarchy. These robots are integrated into standard computing racks with high-density patch panels, enabling seamless adoption in modern datacenters.

The software layer (§V) provides essential resource management capabilities and facilitate seamless integration of *R2D2* into the datacenter’s ecosystem. The *R2D2* software runtime hierarchy spans from the datacenter hypervisor resource allocation and network reconfiguration down to embedded robotic controllers. Its core function is to jointly manage task placement and network reconfiguration. The system’s scheduling algorithm (§V-C) preferentially assigns workloads to already-connected nodes and incorporate the cost of physical reconfiguration into allocation fitness calculation. As a result, this policy actively engineers traffic to enhance its spatial sparsity and temporal stability, minimizing the radix of NIC ports and frequency of robotic movements. When topology changes are needed, high-level commands are delegated to dedicated robot controllers on embedded boards, which manage the low-level motion planning, closed-loop control, and fault handling. This hierarchical separation of the runtime design ensures the system remains scalable and computationally tractable.

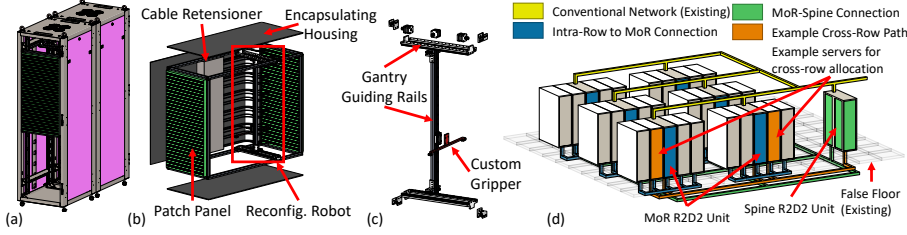


Fig. 5: (a) R2D2 hardware overview in rack chassis. (b) R2D2 unit exploded diagram. (c) Gantry robot internals (d) R2D2 fabric integrated into existing datacenter grid layout.

IV. R2D2 HARDWARE ARCHITECTURE

The R2D2 architecture is a reconfigurable network fabric that uses commodity Cartesian (gantry) robots to dynamically establish direct optical fiber connections between nodes. The R2D2 architecture is built from individual R2D2 units (§IV-A). Each unit is a standalone datacenter rack assembled from off-the-shelf components, including robotic systems repurposed from commodity 3D printers, commercial patch panels, and standard fiber optics. The mature robotics deliver the speed, precision, and energy efficiency required for fine-grained physical network reconfiguration at low cost. Leveraging high-density patch panels and the large build volumes of current 3D printer robots, a single R2D2 unit supports thousands of fiber connections between servers.

The R2D2 fabric is built from multiple R2D2 units and integrates seamlessly with existing datacenter layouts (§IV-B). Each unit serves one of two roles, both using identical hardware: a Middle-of-Row (MoR) unit, positioned centrally within a server row to manage fibers from thousands of NICs and provide full intra-row connectivity, or a Spine unit, which interconnects MoR units across rows to deliver datacenter-scale connectivity and load balancing. The MoR supports even the highest-density server deployments, while the Spine aggregates inter-row links for efficient resource utilization.

In the remainder of this section, we present the detailed design of the R2D2 unit (§IV-A) and its integration into existing datacenter infrastructure (§IV-B), and highlight the key benefits of the R2D2 architecture (§IV-C).

A. R2D2 Unit

Each R2D2 unit consists of four key components: (1) an *encapsulation housing* that encloses and protects the robotic and cabling components, fitting within a standard datacenter rack (Fig. 5). (2) two *patch panel* that expose network cable termination points for both server NICs and inter-unit connections, (3) a *cable retensioning* mechanism that manages network cable slack, and (4) a *gantry-based robot* that physically establishes and removes network links.

Encapsulation Housing: The housing is a standard datacenter rack-mount chassis, with patch panels front and back, and a cable retensioner and reconfiguring robot inside. The housing seals the cable connectors from datacenter airflow and attenuates ambient vibration from surrounding servers, thereby reducing contact face scraping and contamination.

Front and Back Patch Panels: The patch panels host

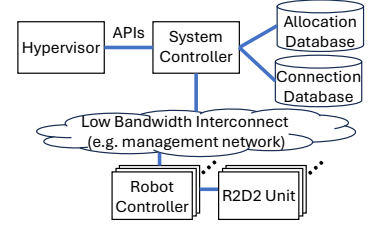


Fig. 6: The integration of R2D2 software into datacenter stack.

receptor ports in a dense 48×96 rectangular array on the front and back of the R2D2 unit. Exterior ports connect directly to server NICs, while interior ports form a grid accessible to the reconfiguring robot. The panel layout is co-designed with the robot's movement plane to ensure every port lies within its operational envelope.

Cable Retensioner: The cable retensioner is a reservoir of slack cables with a constant-force spring reel to retract unused length for connected cables and to fully retract disconnected cables during reconfiguration. This helps prevent entanglement [52] or collision with robotic movements.

Reconfiguring Robot: The R2D2 reconfiguring robot is built using enhanced commodity Cartesian gantry systems and comprises three main components: (1) **the gripper**, (2) **the gantry and guiding rails**, and (3) **the linear drive module**. To establish a connection, the robot retrieves an optical cable from the retensioner, aligns its connector with the target receptor on the patch panel, and inserts it to complete the link between server NICs. Its motion is constrained to a 2D plane, with linear guides and stepper motors enabling precise, repeatable positioning across the receptor grid.

The core modification to commodity gantry systems is a custom gripper - a modular end-effector optimized for connectors. It incorporates self-alignment features to tolerate minor misalignments and reduce wear, integrated sensors to verify latch engagement, and an optional miniature servo to actuate the release tab for unplugging. Once latched, the link is passive, allowing the robot to continue with other reconfigurations. The design targets low-cost scalability, using simple control logic on a lightweight microcontroller. Material-based optimizations can further simplify the gripper; for example, our physical prototype (§VI-B) eliminates the servo by leveraging the natural flex of 3D-printed material.

Because individual R2D2 units are architected for independent execution, we can natively exploit algorithmic-level parallelism across multiple units. We detail this orchestration strategy in the software runtime section (§V).

B. R2D2 Fabric

The R2D2 fabric is a direct-connect network that integrates seamlessly into existing datacenters. Built from high-radix, rack-compatible R2D2 units, it provides direct, single-hop connectivity for thousands of nodes without disturbing existing network infrastructure. Fig. 5(c) illustrates its integration into a standard layout.

R2D2 Middle-of-Row (MoR) Unit The R2D2 MoR Unit is a R2D2 unit centered at each server rack row, connecting to all servers in the row to minimize cable length and complexity. We adopt a centralized MoR design over a distributed ToR model to maximize intra-row connectivity. This enables any compute node to connect to any memory node within the row, independent of physical rack placement.

All R2D2 units – including MoR – are high-radix, reducing fabric hierarchy and enabling dense disaggregation. It can provide up to $48 \times 48 \times 2 = 4608$ ports across front and back panels, all within 42U racks, shown in Fig. 5(b). This enables a single unit to serve up to 27 high-density racks (4 servers/U, 168 servers/rack), 4536 servers. For fault tolerance or additional bandwidth, servers may connect to multiple MoR units, with multi-path routing handled in R2D2 software.

R2D2 Spine Unit To scale beyond a single row, R2D2 uses Spine units that interconnect MoR units to form a datacenter-wide fabric. Spine units leverage unused MoR ports to enable cross-row, global compute-memory sharing and reduce resource stranding. Like MoR units, Spine units offer a high radix of 4608 ports, enabling fabric scaling to over 10 million servers with full bandwidth.

C. Key Benefits of R2D2 Hardware Architecture

Cost and Scalability R2D2 offers low cost and near-linear cost scalability compared to conventional network architectures. Leveraging off-the-shelf gantry robots from the 3D printing market reduces per-port cost to $\sim \$10$, a fraction of the $\sim \$400$ for modern 400 GbE switches. Its high radix enables scaling with at most two levels hierarchy for a datacenter-scale fabric. A single R2D2 MoR unit can support up to 4608 servers, while a 2-tier MoR-Spine topology can scale to over 10 million. Traditional fat-tree networks scale superlinearly in cost; expanding from 512 to 8192 servers with 32-port switches requires moving from a 2-level (1024 cables, 48 switches) to a 3-level fabric (24,576 cables, 1280 switches), increasing cable and switch costs per server by 50% and 67%.

Low Energy R2D2 delivers energy advantage over conventional electronic switches by using passive fiber connections and only consuming power for on-demand connectivity changes. Unlike conventional switches that perform per-packet per-hop forwarding with always-on ASICs and line cards, R2D2 maintains connections through passive physical links, drawing no power during steady-state operation.

Each R2D2 unit consumes under 10 W when idle and peaks at approximately 120 W during reconfiguration. In contrast, a 64-port 400 Gbps switch draws 2.5 kW at peak load and still consumes hundreds of watts under light load; even an idle line card can draw 70 W. By reconfiguring network topology only on demand, R2D2 eliminates the continuous energy overhead of active switching, reducing power and cooling demands.

Low Latency and High Bandwidth R2D2 provides network nodes with direct fiber connections, eliminating the latency and bandwidth bottlenecks from network switching. By bypassing intermediate network hops, R2D2 avoids port-to-port forward latency of microseconds in network switches,

governed only by network software, transceiver delays, and signal propagation speed on network cables. For example, R2D2 endpoints can enjoy full 400 Gbps on a 400 Gbps link. In contrast, switch-based architectures like fat-trees suffer from flow collisions and congestion, especially under incast patterns, reducing effective throughput.

Fault Isolation. A fundamental architectural benefit of R2D2 is the strict physical decoupling of the robotic control plane from the optical data plane. Once a circuit is established, the gantry robot physically detaches from the latched link, leaving an entirely passive optical data path. Because the robots remain completely idle when there are no active re-configuration tasks, mechanical hardware faults cannot disrupt steady-state operations. Instead, failures manifest exclusively during new VM allocations or topology updates, translating merely into slightly increase allocation latency. We detail how the software runtime mitigates these faults—such as failing over to an alternate robot to ensure active tenant workloads remain completely undisturbed—in §V-D. We provide additional evaluation on the impact of allocation latency in §VII, using failure rates derived from empirical field data to model real-world operational conditions.

V. R2D2 SOFTWARE RUNTIME

The R2D2 software runtime performs joint orchestration of the robotic reconfiguration of R2D2 hardware and allocation of tasks to co-optimize allocation latency, resource utilization, and network traffic locality. The R2D2 runtime (Fig. 6) has two hierarchical levels: the *system controller* (§V-A), operating at the datacenter orchestration level, and the *robot controller* (§V-B), managing individual robots.

The R2D2 software engineers resource traffic to promote spatial sparsity and temporal stability with the joint resource allocation and network reconfiguration algorithm (§V-C). The algorithm identifies feasible placements and validates physical connectivity between nodes, while minimizing robotic reconfiguration to reduce task placement latency.

To ensure compatibility with existing datacenter orchestration stacks, the system controller exposes standard APIs, such as Azure Protean rules [32], allowing higher-level orchestrators to request resource allocations and network configurations without modifying their orchestration logic.

A. R2D2 System Controller

The R2D2 system controller orchestrates datacenter-wide resource management across the R2D2 fabric. It selects optimal compute and memory placements, validates resource mappings, and ensures physical connectivity via the robotic network, guided by the joint planning algorithm in §V-C.

Placement requests from upper-layer orchestrators (e.g., hypervisors or schedulers) are sent to the system controller via standard APIs, each specifying compute and memory requirements. The controller invokes the joint algorithm to minimize stranded resources and reconfiguration costs, considering current topology and resource availability. If reconfiguration is required, it issues low-level commands to robot

Algorithm 1 R2D2 Joint Allocation and Reconfiguration

```
1: function ALLOCATETASK(Task  $t$ , Compute req  $C$ , Mem req  $M$ )
2:    $row \leftarrow \text{MAXFITNESS}(\text{FEASIBLEROWS}(C, M))$ 
3:    $c[] \leftarrow \text{SORT}(\text{FEASIBLECNODESNORECONF}(C, M, row))$ 
4:   for  $i = 1$  to  $n$  do
5:     if  $\text{COMMIT}(c[i], t)$  then
6:       return SUCCESSFUL
7:   end for
8:    $p[] \leftarrow \text{SORT}(\text{FEASIBLECMPAIRS}(C, M, row))$ 
9:   for  $i = 1$  to  $n$  do
10:     $r[] = \text{AVAILABLEROBOTS}(p[i])$ 
11:    for  $j = 1$  to  $m$  do
12:      if  $\text{RECONFIGURENETWORKBYROBOT}(r[j], p[i])$  then
13:        if  $\text{COMMIT}(p[i], t)$  then
14:          return SUCCESSFUL
15:        else
16:           $\text{MARKROBOTASDOWN}(r[j])$ 
17:        end if
18:      end for
19:    end for
20:  return QUEUE FOR RETRY
21: end function
```

controllers (e.g., disconnect port A , connect port B). Upon successful reconfiguration, the controller returns the final node assignments to the hypervisor for task scheduling.

B. R2D2 Robot Controller

Each R2D2 robot controller manages a single physical robot and operates on an embedded controller mounted on the robot, offering low-latency, high-fidelity control. The robot controller translates high-level reconfiguration commands into executable motion plans and performs closed-loop control to ensure correctness. Meanwhile, the R2D2 robot controller also actively monitors the R2D2 units' health through a number of metrics such as temperature, motor current, etc.

Upon receiving a reconfiguration request, the controller computes an optimal trajectory for its stepper motors, plans Cartesian motion over the patch panel, and generates low-level actuation instructions (e.g., G-code). It monitors feedback during actuation (e.g., encoder position, insertion force) to ensure precise, collision-free cable manipulation.

After successful reconfiguration (e.g., cable latched), the robot controller notifies the system controller to continue the task placement. On an execution error (e.g., stall, misalignment) or an observed unhealthy (e.g., overcurrent, overheating) hardware state, it performs local retries. If retries fail beyond a threshold, it reports failure and flags its corresponding R2D2 unit unavailable to the system controller. The system controller excludes the unit from future reconfigurations and initiates maintenance/failover procedures.

C. R2D2 Joint Allocation and Reconfig. Algorithm

The R2D2 joint task allocation and network reconfiguration algorithm jointly optimizes task placement and dynamic network reconfiguration, in contrast with existing topology-aware disaggregated resource scheduler [31], [57], [61] that optimizes task placement on statically provisioned topology. Its objective is to promote network sparsity and temporal stability by minimizing reconfigurations, while ensuring low allocation latency and high resource utilization.

To achieve scalability, the algorithm follows a two-stage hierarchical process (Alg. 1): selecting a datacenter row,

followed by intra-row compute-memory node selection. It first identifies feasible rows with sufficient aggregate resources, then selects the one with the highest fitness score under a best-fit policy. Within the chosen row, the algorithm prioritizes compute nodes that meet task requirements using existing memory links to avoid reconfiguration. Candidates are ranked by a fitness function that considers resource fit and link utilization, promoting balanced traffic and scalable fanout. If no such placement exists, it searches all compute-memory pairs in the row, now allowing reconfiguration, and selects a valid placement using the same fitness metric. It iteratively tries different compute-memory pairs and different robots that are available to connect between the pairs. If none is successful, the task is queued for retry. By favoring existing connections, the algorithm reduces topology changes, promoting network sparsity and stability. Reconfiguration is used only when necessary to sustain utilization.

When reconfiguration is inevitable, particularly under bursty VM allocation demands, R2D2 natively exploits spatial parallelism across its distributed reconfiguration units. The co-optimization algorithm is designed to fully utilize this hardware-level concurrency. As reflected in Alg. 1 (Line 12), it queries a real-time availability list of idle robots for each candidate pair. To avoid localized bottlenecks, the scheduler preferentially selects placements that spatially distribute allocations to distinct, concurrently operating robots across different pods, maximizing parallel execution.

Crucially, the R2D2 system controller and the individual robot controllers operate under an asynchronous dispatch model. Once placement decisions are dispatched to the target robots, the system controller immediately proceeds to evaluate subsequent VM allocations without blocking. Because the software control path is strictly decoupled from the physical actuation phase, multiple robots across the system independently and simultaneously execute their assigned tasks. This decentralized execution ensures that system-wide reconfiguration latency scales gracefully, preventing the system controller from becoming a serialization bottleneck.

D. Hardware Failure Handling by R2D2 Software

R2D2's mechanical components are designed for long service lifetimes. Nonetheless, the system incorporates graceful degradation mechanisms modeling after occasional hardware failures of individual R2D2 units and robot controllers.

R2D2 applies a hierarchical failure-management model: **(1) Unit faults outside reconfiguration.** Failures outside active reconfiguration fall into two categories. First, if a R2D2 gantry robot fails while idle, the robot controller reports the fault to the system controller (§V-B). Because connectors are passively latched, existing links remain mechanically secured. The system controller then marks the unit unhealthy, excludes it from future reconfigurations, and schedules it for service. Second, if a link fails after it has already been established, the local R2D2 unit controller first attempts a retry over the existing physical path. If this retry fails (e.g., due to persistent physical-link degradation), the R2D2 system controller provisions a

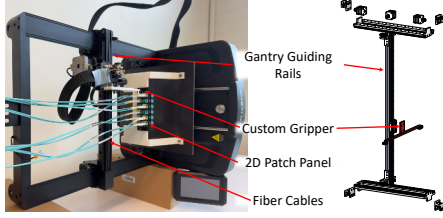


Fig. 7: R2D2 prototype.

local replacement link by allocating spare optical connectors at both target resource nodes and physically routing around the failure transparently to the running workload. Only if both local recovery mechanisms fail does R2D2 fall back to VM live migration or priority reallocation in the next scheduling cycle. (2) **Unit faults during reconfiguration.** Faults during active reconfiguration also fall into two categories. Insertion-related faults (e.g., transient alignment errors) are low risk and are handled through local retry. If the retry fails, or if the unit halts, the R2D2 system controller assigns an alternate robot to complete the same compute-memory-pair configuration. If no alternate robot is available, the scheduler falls back to the next-best feasible placement under the current fabric state. In the worst case, the request is deferred for priority allocation in the next scheduling cycle. By contrast, movement-related anomalies—including abnormal retention force, motor current, or encoder deviation—may threaten existing latched links. In such cases, the controller halts the unit, powers it down, and reports the fault. Only the link currently being reconfigured is affected. The scheduler then assigns an alternate robot to complete the same compute-memory-pair configuration (§V-C), while the faulty unit is queued for service. (3) **Controller-level or multiple failures.** If a robot controller becomes unreachable, the affected R2D2 units are first isolated and their existing links are verified. All links associated with the failed controller are then marked static and excluded from future reconfiguration. Depending on the SLA, the system controller may also migrate migratable tasks away from impaired robots—and, under multiple failures, away from impaired rows—to healthy ones. We detail this live-task-migration methodology in §VII. As with unit-level faults, unreachable controllers are excluded from future scheduling and queued for service.

VI. EVALUATION

We evaluate R2D2 using a combination of small-scale hardware prototyping and datacenter-scale simulation. The prototype validates the mechanical and electrical design, reveals how spec-sheet parameters translate into real hardware behavior, and provides the empirical grounding needed to scale the evaluation to datacenter settings using larger off-the-shelf gantry robots. The datacenter-scale simulations then evaluate R2D2 at both the component level (e.g., how joint allocation-reconfiguration promotes sparsity and locality) and the system level (e.g., task-allocation delay and end-to-end application performance).

A. Experimental Setup

System Setup. We evaluate R2D2 at two scales, 512 and 2048 nodes, using current-generation 400 GbE networks, a practical target for memory disaggregation [57]. Network bandwidth and latency are modeled using a packet-level `ns-3` simulator with 500 ns per-hop latency [71]. Allocation latency is measured using a custom discrete-event simulator exercising the R2D2 runtime and scheduling algorithm. Application-level performance is evaluated using real-world disaggregated application traces [25].

Cost and Power Modeling. We build a parametric model calibrated with prototype cost analysis and power measurements. This enables scaling R2D2’s efficiency to full datacenter deployments and comparison with fat-tree fabrics.

Workload Selection. For network microbenchmark and application-level benchmark, we adopt real-world disaggregated memory traffic traces from Gao et al. [25] and Shoal [62], as well as machine learning workloads with ASTRA-sim [58]. The memory traffic traces comprise 5 workloads: *bdb*, *GraphLab*, *Memcached*, *Terasort_Hadoop* (*Terasort H*), and *Terasort_Spark* (*Terasort S*). Each trace entry encodes a resource access flow between resource nodes with the request time and flow size. Storage traffic are removed to focus on memory disaggregation. As the original trace from [25] contains a fixed number of nodes, we follow the methodology of Shoal [62] for scaling. For ASTRA-sim, we configure the workload after LLaMA 4 Behemoth model [2].

For the task-level microbenchmark, we use the latest cluster of VM traces dataset collected from production datacenters [32], comprising 2064 machines, each equipped with 48 CPU cores and 384GB of memory. Each trace entry contains the requested VM CPU/memory/storage requirements, as well as the duration. We exclude storage and application network requirements to focus on memory disaggregation. The trace includes bursts of 1000s allocation requests per second (Fig. 8(d)). To evaluate R2D2’s behavior under such stress, we analyze the latency of largest spikes in §VI-D.

Architectural Baselines We compare R2D2 against state-of-the-art fat-tree and OCS fabrics. Our fat-tree baseline is NDP [34], which achieves over 95% of optimal fat-tree throughput and represents modern production datacenter practice. For OCS baseline, we use 3D MEMS OCS [38], the technology deployed at scale in Google’s datacenters [55]. Because commercial OCS devices has limited radix (up to 384 ports), we linearly scale cost and power to 576- and 2304-port configurations using published measurements [38], [70], providing a best-case OCS baseline.

Runtime-Algorithm Baselines To evaluate R2D2’s software runtime, we compare R2D2 joint algorithm to a best-fit baseline, a policy widely used in cloud [32], while both scheduling for R2D2 hardware.

B. Prototype Evaluation

To validate the feasibility of robotizing network reconfiguration and to evaluate the physical capabilities and limitations of

Components	Cost	Power	Qty (576P)	Qty (2304P)	Qty (4608P)
42U Server Rack [19]	\$307	0W	1	1	1
96 Fiber Patch Panel [22]	\$109	0W	6	24	48
3m Fiber Patch Cable [21]	\$6.5	0W	288	1152	2304
Elegoo OrangeStorm Giga 3D Printer [47]	\$2499	—	1	1	1
0.8m x 0.8m x 1m Metal Gantry	Incl'd	0W	1	1	1
X-axis: 42HD8022-01	Incl'd	36W	1	1	1
Y-axis: 57HD6436-01	Incl'd	36W	1	1	1
Z-axis: 42HD8011-19	Incl'd	36W	1	1	1
Controller Board	Incl'd	5W	1	1	1
Power Supply	Incl'd	5W	1	1	1
Gripper	≈ \$0	0W	1	1	1
576-Port Total	\$5332	118W	—	—	—
2304-Port Total	\$12910	118W	—	—	—
4608-Port Total	\$23014	118W	—	—	—

TABLE III: Cost and power breakdown of 576-port, 2304-port, and 4608-port R2D2 hardware configurations.

R2D2 hardware—including how spec-sheet parameters translate to real performance—we built a physical prototype by repurposing a commodity 3D printer [46]. We replace the filament feeder and extruder head with a custom 3D-printed gripper designed to latch onto standard LC optical connectors; replace the build plate with a custom 3D-printed mount for a commodity optical patch panel; and reprogram the firmware to enable direct G-code control of the XYZ stepper motors.

Positioning Accuracy and Repeatability The prototype was programmed to reconfigure connectors at random positions over 500 cycles, achieving a 100% cable-connection success rate and a worst-case positional error of 0.02 mm, consistent with the spec sheet rating of the robot.

Reconfiguration Latency We measure end-to-end reconfiguration latency—from request receipt to link completion—across distances. The results match our estimates derived using the methodology in §III-A0b based on spec-sheet parameters, validating our feasibility study. Its effect on task-allocation latency is analyzed in §VI-D.

Power Consumption During idle operation (all motors disabled, heating pad removed), the prototype’s controller, power supply, and sensor draw less than 10 W from the wall (measured via an AC power meter) and peak at 30 W during reconfiguration. This matches the motor spec (3 motors, 5 W each) and informs our energy estimation in §VI-C.

Scaling to Rack-Scale We do **not** directly extrapolate prototype measurements to rack-scale R2D2 hardware, recognizing that large systems introduce added complexity. Instead, we use the prototype measurements to model performance behavior from spec-sheet parameters. Specifically, we use a larger commodity 3D printer [47] with a rack-scale $0.8 \times 0.8 \times 1.0$ m motion range; this printer employs kinematics, drive electronics, and connector mechanics similar to our prototype, making the prototype’s spec-sheet-to-performance translation applicable to rack-scale deployments.

C. Cost and Power Efficiency

As shown in Table III, a 576-port R2D2 unit costs \$5.4K and consumes up to 120 W, while 2304-port and 4608-port units cost \$13K and \$23K with the same peak power.

In the overall network fabric (Fig. 8(a)), R2D2 reduces cost by 23–38% and power by 80% at 512 nodes, and by 23–38% and 84% at 2048 nodes, compared to the fat-tree baseline. R2D2 reduces cost by 24–34% and power by 1–3% at 512 nodes, and by 25–35% and 3–6% at 2048 nodes, compared to

the OCS baseline. These savings grow with scale, highlighting R2D2’s scalability. R2D2 units (including robots) account for 2.1–4.0% of the network fabric power at 512 nodes and only 0.5–1.0% at 2048 nodes. This power efficiency is due to 1) passive single-hop connections drawing power only during reconfigurations, and 2) a high radix that amortizes each robot’s power over 100s to 1000s of nodes, unlike traditional switches serving only 10s. At 8192 nodes (Table VI), R2D2’s cost (36–48%) and power (87%) savings over fat-tree is higher than those of 512 and 2048 nodes, showing better scalability than the current leaf-spine network and three-tier network. At 100G 512-nodes (Table VII), R2D2 maintains cost (42–44%) and power (67–68%) savings. More details in the appendix.

D. Microbenchmarks

Network Evaluation Fig. 8(b)–(c) shows that R2D2 improves average and tail FCT by up to 43.3% and 18.5%, and average and tail throughput by up to 70.2% and 69.4%, against the SOTA fat-tree baseline design at both 512- and 2048-node scales, demonstrating scalable and consistent improvements in both latency and throughput. R2D2 achieves the same latency and throughput as OCS.

Task Allocation Evaluation R2D2’s scheduling latency has only a marginal effect on overall performance. Although absolute latencies are higher: 41–51% (avg) and 27–30% (p99) at 512 nodes, and 5–39% (avg) / 21–29% (p99) at 2048 with spine enabled (Fig. 8(e)–(f)), compared to the fat-tree/OCS baselines, it increases only 0.49% of total VM runtime, as VM execution time dominates. Across all tests, utilization reaches 99% CPU and 69% memory. We do not distinguish between OCS and fat-tree in Fig. 8(e)–(f) as they are within 0.5%.

We analyze the largest task-allocation spikes and find that robotic delay does not amplify them. These spikes incur <15% higher latency than baseline, which is lower than the average allocation-delay increase reported earlier. This is because most spike requests are small, do not require a robotic reconfiguration, and reconfigurations occur in parallel.

Our design-space exploration of existing commodity 3D-printer robots shows that increasing travel speed yields diminishing marginal returns on performance: even a $2\times$ speedup only reduces task completion time overhead by < 0.01%, a negligible improvement offset by a disproportional $10\times$ increase in the robot cost (Fig. 8(g)) [44], [45], [47], [48].

To isolate the effectiveness of R2D2’s joint algorithm in promoting sparsity and stability and minimizing reconfigurations, we also compared it with the best-fit baseline on R2D2 hardware and observed that the joint algorithm achieves 10–20 $\times$ lower average and p99 allocation latency at both scales. The best-fit algorithm, agnostic of robotics, triggers excessive reconfigurations and cascades allocation latency. To quantify robotic reconfiguration parallelism, we find that 37–45% of reconfigurations overlap 2+ robots for the 2 and 4 robot setup.

E. Applications

R2D2 reduces network hardware costs by up to 38% and network hardware energy consumption by up to 84%, while

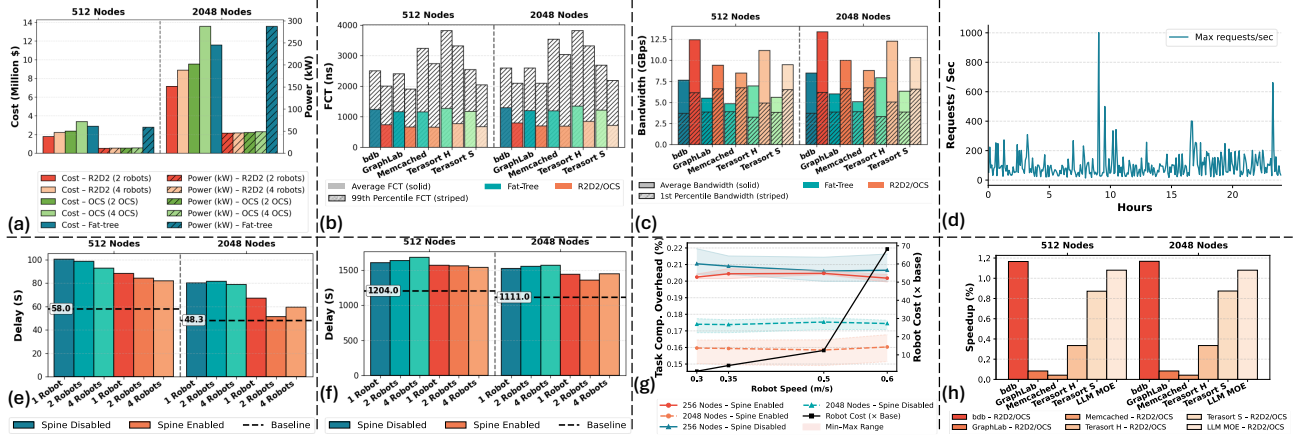


Fig. 8: Results of R2D2 against fat-tree and OCS baselines. (a) Cost and Power. (b) Average and P99 Flow Completion Time (FCT). (c) Average Flow and P01 Flow Throughput. (d) Request Rate for Task Allocation. (e) Average Allocation Latency. (f) P99 Allocation Latency. (g) Sensitivity Analysis on Robot Speed. (h) Normalized Application Performance Speedup.

maintaining application performance comparable to the fat-tree and OCS baselines. As shown in Fig. 8(h), R2D2’s application performance is the same as OCS and a slight speedup over fat-tree for the 6 applications (0.59 % on average for 512 and 2048 nodes). Similarly, our microservice/serverless study shows that R2D2, with widely used optimization techniques [73], experiences only a modest additional allocation latency on average (<2s, 4%) compared to VM workloads.

F. Sensitivity Study

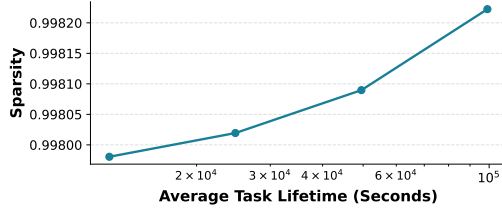


Fig. 9: Sensitivity of link stability to workload lifetime.

To study the operating limit under a diverse range of workloads, we added a sensitivity analysis that sweeps over workload lifetime and arrival rate and measures their impact on the stability of disaggregation traffic patterns. We varied the average workload lifetime from 1.15 days to 3 hours, with a corresponding average arrival rate of 4 to 32 per second and a peak arrival rate of 1K to 8K per second, to match the lifetime of modern short-lived applications such as microservices [73]. We follow the widely used traffic-generation methodology from prior work [62]. As shown in Fig. 9, we found that 1) disaggregation traffic has very small changes (0.44%) in temporal locality when decreasing workload lifetimes from 1.15 days to 3 hours. 2) Because the disaggregation traffic remains stable, R2D2 only experienced a 4s (9%) additional allocation latency when decreasing workload lifetimes from 1.15 days to 3 hours. This can be further reduced to 3s (8%) by scaling the number of robots from 2 to 4. These additional delays are minimal (<0.1%) over the 3-hour lifetime.

VII. DISCUSSION

Availability and Durability While R2D2’s moving components—reconfiguring robots, optical connectors, and cable retensioners - are subject to mechanical wear, their projected lifetimes match or exceed typical datacenter refresh cycles. Commercial 3D-printer motion stages are rated for 7,500-15,000 continuous hours and, under R2D2’s intermittent duty cycle, are projected to last ≥ 10 years. Optical connectors are specified for $\geq 1,000$ mating cycles, while traces indicate only 10-13 per connector per year, providing ample margin for a 4-8-year refresh. Cable retensioners are rated for > 300 full pulls, likewise supporting a ≥ 10 -year horizon. These projections are based on vendor ratings and trace-driven workloads.

Although all components are rated well beyond a typical datacenter refresh cycle, real-world manufacturing defects and field conditions can still produce occasional early-life failures. To quantify their impact, we model mechanical faults by injecting a 4% per-day failure rate (approximately 1.5% of 3D prints require human intervention [54], and more complex multi-filament prints report up to 2.8% [56]). We consider a common datacenter service model: next-day replacement [16], [37] to evaluate recovery latency and system-level impact. R2D2’s fault-handling mechanism limits increases in task allocation latency to only 2.7% and does not impact allocated application performance. Even after accounting for replacement overhead—including human labor (assuming one highly experienced technician working full time on maintaining R2D2 [4], [5], [29])—the total maintenance OpEx remains negligible at just 9.5% of R2D2’s CapEx, with its total cost (CapEx + maintenance OpEx) remains below 85% of the CapEx of a traditional switched network. This localized overhead is far outweighed by the architectural cost savings R2D2 provides relative to traditional switched networks.

We also compare our intervention rate with other existing data center hardware, such as hard drives. Our analysis shows that this intervention burden remains modest, even at data-center scale, and comparable to the existing intervention rate.

For example, a hyperscale deployment with 50,000 servers contains at least 50,000 drives [7]; at a 1–2% annual hard drive disk (HDD) failure rate [66], this implies roughly 500–1,000 HDD failures per year, or about 1.4–2.7 HDD replacements per day. By contrast, supporting the same amount of computer and memory resources using disaggregated resource servers would require about 50 2304-port R2D2 units plus 4 more spine units, and even under a conservative 1.5% per-unit per-day intervention rate, this corresponds to only about 0.81 robot-related human interventions per day across the entire datacenter. Therefore, the expected fleet-wide rate of robot-related maintenance remains well below the routine daily maintenance burden of HDD replacement.

Signal Integrity and Latency. R2D2 reduces intra-rack port-to-port latency by approximately 42% by eliminating ToR-switch traversal. In the worst case (1/2 row), R2D2 adds roughly 10 m of fiber, contributing less than 50 ns of propagation delay; this is small relative to the 455 ns ToR-switch latency. Accounting for roughly 500 ns RNIC latency, the reduction is $1 - \frac{500+50}{500+455} \approx 42.4\%$.

R2D2 achieves error rates comparable to ToR switches because the added optical attenuation remains within the transceiver link budget. The additional cable length and connector insertion add about 0.1 dB of loss, and a scale-out path across two MoR units and one spine adds at most about 0.3 dB. Both are well below the 1.9 dB budget of 400GBASE-SR8 10 m transceivers, so the impact on error rate is negligible.

Non-VM Tasks Virtual machines are the underlying infrastructure for various services such as direct VM allocation, containers, and serverless workloads [32]. By supporting VM-based allocation, R2D2 provides a foundation that can be easily extended to support these non-VM tasks. We leave the exploration of such extensions to future work.

Resource Fragmentation While R2D2 does not directly address the reduction of resource fragmentation, it facilitates resource disaggregation to reduce fragmentation. In task-level microbenchmarks, it averages $\geq 99\%$ compute and $\geq 69\%$ memory utilization, showing low fragmentation.

Cable Entanglement R2D2 avoids cable entanglement by fully retracting the cable with a retensioner between each reconfiguration, following the methodology in [52]. This method has been validated on prototypes, avoiding entanglement under arbitrary cross-connections over extended operation.

VM Migration R2D2 can support VM live migration by treating migration as a standard allocation event with a placement constraint on a resident resource node. For example, a CPU-only migration is treated as a resource allocation for a new CPU node, while the VM’s memory remains on its current memory node. The R2D2 system controller then finds a feasible CPU node that can connect to that memory node. Before migration, R2D2 establishes all required links so VMs never experience partial connectivity during migration.

R2D2 supports three migration cases. (1) For CPU-node-only migration, the allocator provisions a new CPU node and connects it to the pinned memory node; the hypervisor transfers CPU state and releases the old CPU node after

verification. (2) Memory-node-only migration is symmetric to that of CPU-node-only migration. (3) Combined CPU/memory migration is handled as a sequence of the two single-node migrations. If no feasible resource node is available, the controller defers migration until resource became available, rather than disrupting the running VM.

Complex Topologies/Multi-Path Routing Because R2D2 is connection-oriented, it supports arbitrary topologies. There are no architectural limitations on communication patterns, provided the endpoints have sufficient transceiver fanout.

For one-to-many communication (e.g., shared storage), R2D2 provisions concurrent circuits from a storage node to multiple clients. For many-to-many patterns (e.g., GPU training), it provisions job-specific structures—such as direct circuits for sparse collectives (e.g., rings, trees)—obviating the need for a full all-to-all physical mesh. R2D2 derives these target graphs from existing topology-optimization frameworks by updating the controller’s objective function. Furthermore, because these workloads are highly stable, reconfiguration overhead is easily amortized, relaxing robot speed constraints.

Similarly, multi-path routing in a switchless fabric is implemented natively by provisioning parallel point-to-point optical circuits between endpoints, entirely bypassing in-network packet forwarding. The number of parallel circuits is limited only by port availability. This programmable connectivity allows runtimes to explicitly request parallel paths for bandwidth aggregation, failover redundancy, or strict traffic isolation.

VIII. RELATED WORKS

Network Switches. A substantial body of prior work has explored alternative network switch technologies for data-center networks to enhance scalability, cost-efficiency, and performance [11], [27], [33], [50], [55], [62], which can be broadly grouped into software-defined and circuit switches.

Software-defined packet switches: Software-defined packet switches (e.g. Intel Tofino [10], [20], [53]) have been explored to replace traditional fixed-function switches. However, their flexibility is confined to the L2/L3 switching and routing logic; the underlying network hardware remains a statically provisioned, all-to-all topology with fine-grained per-packet switching that cannot adapt to the spatial sparsity of disaggregation traffic. As a result, for traffic with high spatial and temporal locality, only a subset of the hardware capability (e.g., switch packet-forwarding and switching bandwidth) is actually utilized, while the system still pays the cost of topology-level overprovisioning. Compared with same-generation non-programmable switches, software-defined switches cost roughly 50% more [10], [23]. They consume similar switch-chip power [23], [53], but still incur substantial static transceiver power (hundreds of watts per lightly loaded switch), additional per-packet processing/forwarding overhead, and hundreds of nanoseconds of extra packet latency [20]. In contrast, although R2D2’s mechanical components have higher per-component failure rates than the electronic motherboards used in software-defined switches, its aggregate annual hardware failure rate at datacenter scale

remains $3.3\times$ lower than deployed datacenter hard drives, in addition to its hierarchical fault-tolerance mechanisms (§V-D) which further limit the application-/system-level impact.

Circuit switches: Optical circuit switches (e.g., RotorNet [50], ProjecToR [27], Firefly [33], Jupiter Evolving [55]) replace traditional spine switches to reduce cost, power, and latency. Others, like Larry [11], use electrical circuit switches for dynamic rerouting and congestion mitigation, while Shoal [62] introduces nanosecond-scale circuit switches for coordination-free scheduling. Nonetheless, these networks are statically provisioned, traffic-agnostic, and all-to-all networks with limited flexibility. Their flexibility is confined to fixed schedule topology changes based on offline demand analyses [27], [33], [50], [55], [62]. Modern OCS designs (e.g., 3D MEMS) rely on non-latching mirrors and incur higher optical attenuation. In contrast, R2D2 uses low-cost commodity robotics and off-the-shelf passive components to realize latching, low-loss direct connections. Because disaggregation traffic remains stable at VM-lifetime timescales, R2D2 can hide its relatively slow mechanical reconfiguration time while still adapting the physical topology to actual workload demand, avoiding the attenuation and active-stabilization drawbacks of OCS while providing substantially finer-grained control over resource connectivity.

Overall, unlike prior work that primarily improves switch hardware within a statically provisioned network fabric, R2D2 rethinks the network as a software-hardware co-designed, demand-driven physical interconnect for resource disaggregation, and is largely orthogonal to these prior approaches.

Networks and Interconnects for Resource Disaggregation. A significant body of prior work on resource disaggregation has focused on reducing disaggregation latency and performance overhead by optimizing host software atop standard networks. Infiniswap [30] and FastSwap [3] use one-sided RDMA to bypass remote CPUs. LegoOS [61] is an RDMA-based OS with features like remote memory coalescing.

Recent systems such as Clio [31] remove control logic and transport protocols from memory nodes, while Pond [41] uses CXL for hardware-assisted access. SMART [59] improves RDMA scalability, and Pulse [65] offloads pointer chasing to memory nodes. In contrast, R2D2 targets the interconnect and its runtime, co-optimizing hardware topology and orchestration. It is complementary to RDMA and memory-node techniques, enabling further latency gains.

Other efforts redesign NICs and protocols to reduce latency and power [28], [62], [64]. Aquila [28] co-designs NIC/ToR hardware with a custom L2 protocol; EDM [64] embeds logic in the PHY to bypass MAC processing. These require invasive stack changes. R2D2, by contrast, is host-transparent and compatible with such hardware. Finally, R2D2 performs fine-grained, demand-driven reconfiguration to capture traffic locality, using commodity robotics instead of custom fast switches, reducing cost and easing deployment.

Robotic-Enhanced Systems. Existing robotic systems in datacenters primarily focused on automating traditional human maintenance tasks [18], [36], [40], [52], [72], rather than reconfiguring the network fabric at runtime to improve

network efficiency. Telescent [40], Mizukami et al. [52] and Xu et al. [72] introduce robotic patch panels and robotic arms for automated cable management. Hong et al. [36] explore autonomous network maintenance via robotic agents, while Elbehery et al. [18] examine broader robotic functions such as equipment handling and environmental monitoring. In addition to targeting fundamentally different objectives than R2D2, all of these systems rely on costly custom-built robots designed for specific tasks. In contrast, R2D2 makes maximal use of mass-produced commodity hardware, enabling cost-effective, scalable, and rapid deployment in datacenters.

Recent proposals have explored mechanical approaches [43], [63], [70] to achieving network configuration or accelerating data movement in data centers. RF-MG [63] leverages physical mobility to improve wireless signal strength. TopoOpt [70] uses robotic patch panels to optimize GPU server communication for distributed ML training (specifically targeting application traffic) based on offline network traffic analysis. DHL [43] physically transports SSDs using magnetic rails to relocate data within the datacenter. R2D2 is orthogonal to these works by targeting disaggregation traffic instead of application traffic or data, exploiting both spatial and temporal locality in traffic patterns by fine-grained, dynamic, on-demand runtime network reconfiguration, and leveraging commodity robotics instead of costly, custom-built components.

IX. CONCLUSION

We have demonstrated R2D2, a reconfigurable network architecture that, for the first time, leverages mature commodity robotics to dynamically establish direct, single-hop, pairwise physical connections between disaggregated CPU and memory nodes in response to changing traffic patterns. This approach effectively addresses the spatial overprovisioning of statically configured all-to-all connections and the temporal overprovisioning of static, continuous per-packet switching inherent in traditional disaggregated data center networks. By co-scheduling task placement with topology reconfiguration, we reduce both cabling complexity and reconfiguration delays, paving the way for the practical deployment of robotized networking in DDCs. Our results show that R2D2 is feasible to deploy and delivers significant cost and power advantages from a network hardware perspective. Meanwhile, from an application standpoint, R2D2 can achieve a comparable or higher performance over the SOTA fat-tree network. While some challenges remain for moving into production, as we discussed in the paper, we believe that most major issues have been addressed, and we leave the remaining ones for future research. Through this case study, we hope to inspire further exploration into network specialization and robotized systems.

X. ACKNOWLEDGEMENT

The authors thank the anonymous reviewers for their valuable comments and suggestions. The authors also thank Dr. Jialiang Zhang, Peter Bruno, and the staff of the AddLab at the University of Pennsylvania for their support and contributions during the early stages of this work.

REFERENCES

- [1] B. Abali, R. J. Eickemeyer, H. Franke, C.-S. Li, and M. A. Taubenblatt, "Disaggregated and optically interconnected memory: when will it be cost effective?" Mar. 2015.
- [2] M. AI, "The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation," <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, 2025.
- [3] E. Amaro, C. Branner-Augmon, Z. Luo, A. Ousterhout, M. K. Aguilera, A. Panda, S. Ratnasamy, and S. Shenker, "Can far memory improve job throughput?" in *Proceedings of the Fifteenth European Conference on Computer Systems*, ser. EuroSys '20. New York, NY, USA: Association for Computing Machinery, Apr. 2020, pp. 1–16.
- [4] Amazon, "Data Center Technician," <https://www.amazon.jobs/en/jobs/3193833/data-center-technician>.
- [5] —, "Senior Data Center Technician," <https://amazon.jobs/en/jobs/3178411/senior-data-center-technician-dec-4-dcc-communities>.
- [6] Amazon Web Services, "Cloud compute instances – amazon ec2 instance types – aws," <https://aws.amazon.com/ec2/instance-types/>, 2025.
- [7] L. A. Barroso, U. Hölzle, and P. Ranganathan, *The Datacenter as a Computer: Designing Warehouse-Scale Machines*, ser. Synthesis Lectures on Computer Architecture. Cham: Springer International Publishing, 2019.
- [8] Broadcom, "P1400gd | single-port 400g pcie ethernet nic," <https://www.broadcom.com/products/ethernet-connectivity/network-adapters/p1400g>, 2024.
- [9] W. Cao, Y. Zhang, X. Yang, F. Li, S. Wang, Q. Hu, X. Cheng, Z. Chen, Z. Liu, J. Fang, B. Wang, Y. Wang, H. Sun, Z. Yang, Z. Cheng, S. Chen, J. Wu, W. Hu, J. Zhao, Y. Gao, S. Cai, Y. Zhang, and J. Tong, "Polardb serverless: A cloud native database for disaggregated data centers," in *Proceedings of the 2021 International Conference on Management of Data*, ser. SIGMOD '21. New York, NY, USA: Association for Computing Machinery, Jun. 2021, pp. 2477–2489.
- [10] CDW, "Edgecore 32 port 40g qsfp-dd switch," 2025. [Online]. Available: <https://www.cdw.com/product/edgecore-32-port-40g-qsfp-dd-switch/7131371>
- [11] A. Chatzileftheriou, S. Legtchenko, H. Williams, and A. Rowstron, "Larry: Practical network reconfigurability in the data center," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, 2018, pp. 141–156.
- [12] X. Chen, L. Yu, V. Liu, and Q. Zhang, "Cowbird: Freeing cpus to compute by offloading the disaggregation of memory," in *Proceedings of the ACM SIGCOMM 2023 Conference*, ser. ACM SIGCOMM '23. New York, NY, USA: Association for Computing Machinery, Sep. 2023, pp. 1060–1073.
- [13] M. Chowdhury, S. Kandula, and I. Stoica, "Leveraging endpoint flexibility in data-intensive clusters," in *Proceedings of the ACM SIGCOMM 2013 conference on SIGCOMM*, ser. SIGCOMM '13. New York, NY, USA: Association for Computing Machinery, Aug. 2013, pp. 231–242.
- [14] CXL Consortium, "Compute express link," <https://computeexpresslink.org/>, 2019.
- [15] C. Delimitrou, S. Sankar, A. Kansal, and C. Kozyrakis, "Echo: Recreating network traffic maps for datacenters with tens of thousands of servers," in *2012 IEEE International Symposium on Workload Characterization (IISWC)*, Nov. 2012, pp. 14–24.
- [16] Dell, "ProSupport One for Data Center," 2025. [Online]. Available: https://www.delltechnologies.com/asset/en-us/services/support/technical-support/prosupport_one_for_data_center_service_overview_data_sheet.pdf
- [17] A. Dragojević, D. Narayanan, M. Castro, and O. Hodson, "Farm: Fast remote memory," in *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, 2014, pp. 401–414.
- [18] K. Elbehri and H. Elbehri, "Overlay robotized datacenter system," in *Proceedings of Eighth International Congress on Information and Communication Technology*. Springer, Singapore, 2023, pp. 1–14.
- [19] R. Electronics, "Raising electronics 42u 4 post server rack open frame enclosure 19" adjustable depth 22-34", <https://risingracks.com/raising-electronics-42u-4-post-open-frame-server-rack-enclosure-19-adjustable-depth-22-34/>, 2025.
- [20] D. Franco, E. Ollora Zaballa, M. Zang, A. Atutxa, J. Sasiain, A. Pruski, E. Rojas, M. Higuero, and E. Jacob, "A comprehensive latency profiling study of the Tofino P4 programmable ASIC-based hardware," *Computer Communications*, vol. 218, pp. 14–30, Mar. 2024.
- [21] FS, "3m (10ft) fiber patch cable, 1 fiber, lc upc simplex to lc upc simplex, multimode (om4), lszh, 1.2mm, tight-buffered, aqua," <https://www.fs.com/products/300743.html>, 2025.
- [22] —, "48 port lc fiber patch panel, om4 multimode," <https://www.fs.com/products/69082.html>, 2025.
- [23] —, "N8610-32d switch datasheet," https://www.fs.com/products/_support.html?isPica=false&categories_id=5125&third_categories_id=4228&products_model=N8610-32D&style_id=15&files_id=7768, 2025.
- [24] —, "N9510-64d switch datasheet," https://www.fs.com/products/_support.html?isPica=false&categories_id=3404&third_categories_id=3255&products_model=N9510-64D&style_id=15&files_id=7808, 2025.
- [25] P. X. Gao, A. Narayan, S. Karandikar, J. Carreira, S. Han, R. Agarwal, S. Ratnasamy, and S. Shenker, "Network requirements for resource disaggregation," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 249–264.
- [26] Gen-Z Consortium, "Gen-z," <https://genzconsortium.org/>, 2016.
- [27] M. Ghobadi, R. Mahajan, A. Phanishayee, N. Devanur, J. Kulkarni, G. Ranade, P.-A. Blanche, H. Rastegarfar, M. Glick, and D. Kilper, "Projector: Agile reconfigurable data center interconnect," in *Proceedings of the 2016 ACM SIGCOMM Conference*, ser. SIGCOMM '16. New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 216–229.
- [28] D. Gibson, H. Hariharan, E. Lance, M. McLaren, B. Montazeri, A. Singh, S. Wang, H. M. G. Wassel, Z. Wu, S. Yoo, R. Balasubramanian, P. Chandra, M. Cutforth, P. Cuy, D. Decotigny, R. Gautam, A. Iriza, M. M. K. Martin, R. Roy, Z. Shen, M. Tan, Y. Tang, M. Wong-Chan, J. Zbiciak, and A. Vahdat, "Aquila: A unified, low-latency fabric for datacenter networks," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, 2022, pp. 1249–1266.
- [29] Google, "Data Center Technician IV — Google Careers," <https://www.google.com/about/careers/applications/jobs/results/108492767394439878-data-center-technician-iv>.
- [30] J. Gu, Y. Lee, Y. Zhang, M. Chowdhury, and K. G. Shin, "Efficient memory disaggregation with infiniswap," in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 649–667.
- [31] Z. Guo, Y. Shan, X. Luo, Y. Huang, and Y. Zhang, "Clio: a hardware-software co-designed disaggregated memory system," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, Feb. 2022, pp. 417–433.
- [32] O. Hadary, L. Marshall, I. Menache, A. Pan, E. E. Greeff, D. Dion, S. Dorminey, S. Joshi, Y. Chen, M. Russinovich, and T. Moscibroda, "Protean: Vm allocation service at scale," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 845–861.
- [33] N. Hamedazimi, Z. Qazi, H. Gupta, V. Sekar, S. R. Das, J. P. Longtin, H. Shah, and A. Tanwer, "Firefly: a reconfigurable wireless data center fabric using free-space optics," in *Proceedings of the 2014 ACM conference on SIGCOMM*, ser. SIGCOMM '14. New York, NY, USA: Association for Computing Machinery, Aug. 2014, pp. 319–330.
- [34] M. Handley, C. Raiciu, A. Agache, A. Voinescu, A. W. Moore, G. Antichi, and M. Wójcik, "Re-architecting datacenter networks and stacks for low latency and high performance," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '17. New York, NY, USA: Association for Computing Machinery, Aug. 2017, pp. 29–42.
- [35] Hewlett Packard Corporation, "The machine: A new kind of computer," <https://www.hpl.hp.com/research/systems-research/themachine/>, 2014.
- [36] F. Hong, I. Sarantopoulos, E. Hogg, D. Richardson, Y. Zhang, H. Williams, D. Sweeney, A. Chatzileftheriou, and A. Rowstron, "Self-maintaining [networked] systems: The rise of datacenter robotics!" in *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*, ser. HotNets '24. New York, NY, USA: Association for Computing Machinery, Nov. 2024, pp. 159–166.
- [37] HPE, "HPE Proactive Care Service – Support Services," 2025. [Online]. Available: <https://www.hpe.com/psnow/doc/4aa3-8855enw.pdf>
- [38] hubersuhner, "POLATIS® 7000 Optical Circuit Switch - HUBER+SUHNER," 2025. [Online]. Available: <https://www.hubersuhner.com/en/shop/product/other-systems/optical->

- switches/rack-mount-circuit-switches/85223159/polatis-7000-optical-circuit-switch
- [39] Intel Corporation, "Intel rack scale design architecture white paper," <https://www.intel.com/content/dam/www/public/us/en/documents/white-papers/rack-scale-design-architecture-white-paper.pdf>, 2018.
 - [40] A. S. Kewitsch, "Large scale, all-fiber optical cross-connect switches for automated patch-panels," *Journal of Lightwave Technology*, vol. 27, no. 15, pp. 3107–3115, Aug. 2009.
 - [41] H. Li, D. S. Berger, L. Hsu, D. Ernst, P. Zardoshti, S. Novakovic, M. Shah, S. Rajadnya, S. Lee, I. Agarwal, M. D. Hill, M. Fontoura, and R. Bianchini, "Pond: Cxl-based memory pooling systems for cloud platforms," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS 2023. New York, NY, USA: Association for Computing Machinery, Jan. 2023, pp. 574–587.
 - [42] K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch, "Disaggregated memory for expansion and sharing in blade servers," in *Proceedings of the 36th annual international symposium on Computer architecture*, ser. ISCA '09. New York, NY, USA: Association for Computing Machinery, Jun. 2009, pp. 267–278.
 - [43] G. López-Paradís, I. M. Hair, S. Kannan, R. Rabbat, P. Murray, A. Lopes, R. Zahedi, W. Zuo, and J. Balkind, "The case for data centre hyper-loops," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2024, pp. 230–244.
 - [44] MatterHackers, "BigRep PRO Industrial Large-Format 3D Printer," <https://www.matterhackers.com/store//bigrep-pro-industrial-large-format-3d-printer>, 2025.
 - [45] —, "Builder3D Extreme 1500 PRO 3D Printer," <https://www.matterhackers.com/store//builder3d-extreme-1500-pro-3d-printer/sk/MSSM848L>, 2025.
 - [46] —, "Creality Ender 3 V3 KE 3D Printer," <https://www.matterhackers.com/store//creality3d-ender-3-v3-ke-3d-printer>, 2025.
 - [47] —, "Elegoo OrangeStorm Giga High-Speed Large-Format 3D Printer Kit Enterprise Bundle," <https://www.matterhackers.com/store//elegoo-orangestorm-giga-high-speed-large-format-3d-printer-kit/sk/MSTCQ8SV>, 2025.
 - [48] —, "Modix BIG-120X V4 3D Printer Kit," <https://www.matterhackers.com/store//modix-big-120x-v4-3d-printer-kit/sk/MTPRRVG8>, 2025.
 - [49] —, "Prusa MK4S 3D Printer - Fully Assembled," <https://www.matterhackers.com/store//prusa-mk4s-3d-printer/sk/MIXK1N88>, 2025.
 - [50] W. M. Mellette, R. McGuinness, A. Roy, A. Forench, G. Papen, A. C. Snoeren, and G. Porter, "Rotornet: A scalable, low-complexity, optical datacenter network," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '17. New York, NY, USA: Association for Computing Machinery, Aug. 2017, pp. 267–280.
 - [51] X. Meng, V. Pappas, and L. Zhang, "Tvmpp: Improving the scalability of data center networks with traffic-aware virtual machine placement," in *2010 Proceedings IEEE INFOCOM*, Mar. 2010, pp. 1–9.
 - [52] M. Mizukami and M. Makihara, "Fiber-handling robot and optical connection mechanisms for automatic cross-connection of multiple optical connectors," *Optical Engineering*, vol. 52, no. 7, p. 076116, Jul. 2013.
 - [53] E. Networks, "DCS810 - Edgecore Networks," 2023. [Online]. Available: <https://www.edge-core.com/product/dcs810/>
 - [54] C. Oriotis, "ReDeTec Prusa Print Farm Report," <https://redetec.com/blogs/blog/redetec-prusa-print-farm-report>, 2023.
 - [55] L. Poutievski, O. Mashayekhi, J. Ong, A. Singh, M. Tariq, R. Wang, J. Zhang, V. Beauregard, P. Conner, S. Gribble, R. Kapoor, S. Kratzer, N. Li, H. Liu, K. Nagaraj, J. Ornstein, S. Sawhney, R. Urata, L. Vicisano, K. Yasumura, S. Zhang, J. Zhou, and A. Vahdat, "Jupiter evolving: Transforming google's datacenter network via optical circuit switches and software-defined networking," in *Proceedings of ACM SIGCOMM 2022*, 2022.
 - [56] J. Průša, "Original Prusa i3 MK3S and MMU2S release, SL1 and powder-coated sheets update," Feb. 2019.
 - [57] Y. Qiao, C. Wang, Z. Ruan, A. Belay, Q. Lu, Y. Zhang, M. Kim, and G. H. Xu, "Hermi: Low-latency, high-throughput, and transparent remote memory via feedback-directed asynchrony," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 181–198.
 - [58] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, "Astra-sim: Enabling sw/hw co-design exploration for distributed dl training platforms," in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020, pp. 81–92.
 - [59] F. Ren, M. Zhang, K. Chen, H. Xia, Z. Chen, and Y. Wu, "Scaling up memory disaggregated applications with smart," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '24, vol. 1. New York, NY, USA: Association for Computing Machinery, Apr. 2024, pp. 351–367.
 - [60] Z. Ruan, M. Schwarzkopf, M. K. Aguilera, and A. Belay, "Aifm: High-performance, application-integrated far memory," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 315–332.
 - [61] Y. Shan, Y. Huang, Y. Chen, and Y. Zhang, "Legoos: A disseminated, distributed os for hardware resource disaggregation," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 69–87.
 - [62] V. Shrivastav, A. Valadarsky, H. Ballani, P. Costa, K. S. Lee, H. Wang, R. Agarwal, and H. Weatherspoon, "Shoal: A network architecture for disaggregated racks," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 255–270.
 - [63] J. M. Smith, M. P. Olivieri, A. Lackpour, and N. Hinnerschitz, "Rf-mobility gain: concept, measurement campaign, and exploitation," *IEEE Wireless Communications*, vol. 16, no. 1, pp. 38–44, Feb. 2009.
 - [64] W. Su and V. Shrivastav, "Edm: An ultra-low latency ethernet fabric for memory disaggregation," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, Mar. 2025, pp. 377–394.
 - [65] Y. Tang, S.-s. Lee, A. Bhattacharjee, and A. Khandelwal, "pulse: Accelerating distributed pointer-traversals on disaggregated memory," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, Mar. 2025, pp. 858–875.
 - [66] D. S. Team, "Backblaze Drive Stats for Q2 2025 | Hard Drive Failure Rates," <https://www.backblaze.com/blog/backblaze-drive-stats-for-q2-2025/>, Aug. 2025.
 - [67] Thinkmate, "Supermicro superserver 222ha-tn (sys-222ha-tn)," <https://www.thinkmate.com/system/superserver-222ha-tn>, 2025.
 - [68] C. Wang, H. Ma, S. Liu, Y. Li, Z. Ruan, K. Nguyen, M. D. Bond, R. Netravali, M. Kim, and G. H. Xu, "Semeru: A memory-disaggregated managed runtime," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 261–280.
 - [69] C. Wang, H. Ma, S. Liu, Y. Qiao, J. Elyofson, C. Navasca, S. Lu, and G. H. Xu, "Memliner: Lining up tracing and application for a far-memory-friendly runtime," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 35–53.
 - [70] W. Wang, M. Khazraee, Z. Zhong, M. Ghobadi, Z. Jia, D. Mudigere, Y. Zhang, and A. Kewitsch, "Topoopt: Co-optimizing network topology and parallelization strategy for distributed training jobs," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 739–767.
 - [71] B. Wheeler, "Tomahawk 4 switch first to 25.6tbps," *Microprocessor Report*, Dec. 2019.
 - [72] X. Xu, H. Chen, M. Scheutzw, J. E. Simsarian, R. Ryf, G. Qua, A. Hande, R. Dinoff, M. Szczerban, M. Mazur, L. Dallachiesa, N. K. Fontaine, J. Sandoz, M. Coss, and D. T. Neilson, "Automated fiber switch with path verification enabled by an ai-powered multi-task mobile robot," *Journal of Optical Communications and Networking*, vol. 16, no. 6, pp. 624–630, Jun. 2024.
 - [73] Y. Zhang, Í. Goiri, G. I. Chaudhry, R. Fonseca, S. Elnikety, C. Delimitrou, and R. Bianchini, "Faster and Cheaper Serverless Computing on Harvested Resources," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, ser. SOSP '21. New York, NY, USA: Association for Computing Machinery, Oct. 2021, pp. 724–739.
 - [74] Y. Zhang, C. Ruan, C. Li, X. Yang, W. Cao, F. Li, B. Wang, J. Fang, Y. Wang, J. Huo, and C. Bi, "Towards cost-effective and elastic cloud database deployment via memory disaggregation," *Proc. VLDB Endow.*, vol. 14, no. 10, pp. 1900–1912, Jun. 2021.
 - [75] Y. Zhou, H. M. G. Wassel, S. Liu, J. Gao, J. Mickens, M. Yu, C. Kennelly, P. Turner, D. E. Culler, H. M. Levy, and A. Vahdat,

APPENDIX

In this appendix, we construct fat-tree, OCS, and R2D2 networks to compare their cost and power consumption.

For the 512-node system, we configure a 400GbE fat-tree using Broadcom P1400GD 400 Gb NICs [8] and N8610-32D switches with 32×400 Gb ports [23]. For the 2048-node system, we instead employ N9510-64D switches with 64×400 Gb ports [24], while retaining the same NICs. For the 8192-node 3-tier system, we use the same NICs and switches as the 512-node system. For the 100G 512-node system, we use Mellanox ConnectX-5 100 Gb NICs and N8560-32C switches with 32×100 Gb ports. In all cases, copper cables connect servers to top-of-rack switches, and optical links connect switches.

For comparison, we construct equivalent 2-robot and 4-robot R2D2 fabrics with identical NICs. Switches are replaced by R2D2 units interconnected entirely via optical links. To interface multiple robots with a single NIC, the 2-robot fabric uses 400G-to- 2×200 G breakouts, while the 4-robot fabric uses 400G-to- 4×100 G breakouts.

For OCS baseline, we construct equivalent 2-OCS and 4-OCS fabrics with identical NICs. R2D2 units are replaced by OCS. Because state-of-the-art MEMS OCS are limited to 384 ports, we scale pricing and power from latest 384-port data [38], [70].

Table IV, Table V, and Table VI summarize the 400G cost and power results. At 512 nodes, the fat-tree costs USD 2.89 M and consumes 59.4 kW, the OCS costs USD 2.39 M (2-OCS) / USD 3.41 M (4-OCS) and consumes 11.8 kW / 12.2 kW, compared to USD 1.80 M (2-robot) / USD 2.24 M (4-robot) and 11.6 kW / 11.9 kW for R2D2. R2D2 achieves 38% (2-robot) and 23% (4-robot) lower cost, and roughly 80% lower power compared to fat-tree, and 24.6% (2 units) and 34.5% (4 units) lower cost, and 1.4% (2 units) and 2.6% (4 units) lower power compared to OCS. At 2048 nodes, the fat-tree costs USD 11.5 M and consumes 288 kW, the OCS costs USD 9.54 M (2-OCS) / USD 13.6 M (4-OCS) and consumes 47.4 kW / 48.9 kW, whereas R2D2 requires USD 7.18 M / 8.90 M and 45.9 kW / 46.2 kW. R2D2 achieves 38% (2-robot) and 23% (4-robot) cost reduction with 84% lower power compared to fat-tree, and 24.9% (2 units) and 34.7% (4 units) lower cost, and 2.8% (2 units) and 5.6% (4 units) lower power compared to OCS. At 8192 nodes, the fat-tree costs USD 55.8 M and consumes 1.46 MW, the OCS costs USD 50.1 M (2-OCS) / USD 78.5 M (4-OCS) and consumes 197 kW / 211 kW, whereas R2D2 requires USD 28.8 M / 35.9 M and 185 kW / 187 kW. R2D2 achieves 48% (2-robot) and 36% (4-robot) cost reduction with 87% lower power compared to fat-tree, and 42% (2 units) and 54% (4 units) lower cost, and 6% (2 units) and 12% (4 units) lower power compared to OCS. Table VII summarize the 100G 512-node cost and power results.

Notably, R2D2 units contribute only a small fraction of the overall network, indicating low sensitivity to individual robot cost and power. At 512 nodes, R2D2 robotic units account for just 0.6–1.0% of total cost and 2.1–4.0% of power, dropping further to 0.4–0.6% of cost and 0.5–1.0% of power at 2048 nodes. In contrast, network switches dominate fat-tree networks, contributing 31.3% of cost and 80.8% of power at 512 nodes, and 31.1% and 84% respectively at 2048 nodes. Meanwhile, OCS units contributes significant network cost (25–35%) but minor power consumption (3.4–6.5%).

Components	512-node Fat-Tree			2-OCS			4-OCS			2-Robot R2D2			4-Robot R2D2		
	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty
N8610-32D 32 x 400Gb switch	18.8K	1000	48	—	—	—	—	—	—	—	—	—	—	—	—
OCS Unit (576 Ports)	—	—	—	~300K	~200	2	~300K	~200	4	—	—	—	—	—	—
R2D2 Unit (576 Ports)	—	—	—	—	—	—	—	—	—	5.4K	120	2	5.4K	120	4
Broadcom P1400GD 400Gb NIC	2198	22.3	512	2198	22.3	512	2198	22.3	512	2198	22.3	512	2198	22.3	512
Copper 400Gb Cable (2m)	169	Incl. in NIC	512	—	—	—	—	—	—	—	—	—	—	—	—
Optical Transceivers	749	Incl. in NIC	1024	—	—	—	—	—	—	—	—	—	—	—	—
Optical 400G to 2x200G Transceivers	—	—	—	1293	Incl. in NIC	512	—	—	—	1293	Incl. in NIC	512	—	—	—
Optical 400G to 4x100G Transceivers	—	—	—	—	—	—	2123.5	Incl. in NIC	512	—	—	—	2123.5	Incl. in NIC	512
Optical Cables (3/5/10m)	10	Incl. in NIC	512	—	—	—	—	—	—	—	—	—	—	—	—
Optical Cables (10m)	—	—	—	Incl'd	Incl. in NIC	1024	Incl'd	Incl. in NIC	2048	Incl'd	Incl. in NIC	1024	Incl'd	Incl. in NIC	2048
Total	2.89M	59.4K		2.39M	11.8K		3.41M	12.2K		1.80M	11.6K		2.24M	11.9K	

TABLE IV: Comparison of cost and power components for a 400G 512-node fat-tree, a 2-OCS network, a 4-OCS network, a 2-robot R2D2 fabric, and a 4-robot R2D2 fabric. OCS pricing and power scaled from [38], [70]. Network hardware pricing from FS.com.

Components	2048-node Fat-Tree			2-OCS			4-OCS			2-Robot R2D2			4-Robot R2D2		
	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty
N9510-64D 64 x 400Gb switch	37.3K	2524	96	—	—	—	—	—	—	—	—	—	—	—	—
OCS Unit (2304 Ports)	—	—	—	~1.2M	~800	2	~1.2M	~800	4	—	—	—	—	—	—
R2D2 Unit (2304 Ports)	—	—	—	—	—	—	—	—	—	13K	120	2	13K	120	4
Broadcom P1400GD 400Gb NIC	2198	22.3	2048	2198	22.3	2048	2198	22.3	2048	2198	22.3	2048	2198	22.3	2048
Copper 400Gb Cable (2m)	169	Incl. in NIC	2048	—	—	—	—	—	—	—	—	—	—	—	—
Optical Transceivers	749	Incl. in NIC	4096	—	—	—	—	—	—	—	—	—	—	—	—
Optical 400G to 2x200G Transceivers	—	—	—	1293	Incl. in NIC	2048	—	—	—	1293	Incl. in NIC	2048	—	—	—
Optical 400G to 4x100G Transceivers	—	—	—	—	—	—	2123.5	Incl. in NIC	2048	—	—	—	2123.5	Incl. in NIC	2048
Optical Cables (3/5/10m)	10	Incl. in NIC	2048	—	—	—	—	—	—	—	—	—	—	—	—
Optical Cables (10m)	—	—	—	Incl'd	Incl. in NIC	4096	Incl'd	Incl. in NIC	8192	Incl'd	Incl. in NIC	4096	Incl'd	Incl. in NIC	8192
Total	11.5M	288K		9.55M	47.3K		13.6M	48.9K		7.18M	45.9K		8.90M	46.2K	

TABLE V: Comparison of cost and power components for a 400G 2048-node fat-tree, a 2-OCS network, a 4-OCS network, a 2-robot R2D2 network, and a 4-robot R2D2 network. OCS pricing and power scaled from [38], [70]. Network hardware pricing from FS.com.

Components	8192-node Fat-Tree			2-OCS			4-OCS			2-Robot R2D2			4-Robot R2D2		
	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty
N8610-32D 32 x 400Gb switch	18.8K	1000	1280	—	—	—	—	—	—	—	—	—	—	—	—
OCS Unit (2304 Ports)	—	—	—	~1.2M	~800	18	~1.2M	~800	36	—	—	—	—	—	—
R2D2 Unit (2304 Ports)	—	—	—	—	—	—	—	—	—	13K	120	18	13K	120	36
Broadcom P1400GD 400Gb NIC	2198	22.3	8192	2198	22.3	8192	2198	22.3	8192	2198	22.3	8192	2198	22.3	8192
Copper 400Gb Cable (2m)	169	Incl. in NIC	8192	—	—	—	—	—	—	—	—	—	—	—	—
Optical Transceivers	749	Incl. in NIC	16384	—	—	—	—	—	—	—	—	—	—	—	—
Optical 400G to 2x200G Transceivers	—	—	—	1293	Incl. in NIC	512	—	—	—	1293	Incl. in NIC	8192	—	—	—
Optical 400G to 4x100G Transceivers	—	—	—	—	—	—	2123.5	Incl. in NIC	8192	—	—	—	2123.5	Incl. in NIC	8192
Optical Cables (3/5/10m)	10	Incl. in NIC	8192	—	—	—	—	—	—	—	—	—	—	—	—
Optical Cables (10m)	—	—	—	Incl'd	Incl. in NIC	16384	Incl'd	Incl. in NIC	32768	Incl'd	Incl. in NIC	16384	Incl'd	Incl. in NIC	32768
Total	55.8M	1.46M		50.1M	197K		78.5M	211K		28.8M	185K		35.9M	187K	

TABLE VI: Comparison of cost and power components for a 400G 8192-node fat-tree, a 2-OCS network, a 4-OCS network, a 2-robot R2D2 network, and a 4-robot R2D2 network. OCS pricing and power scaled from [38], [70]. Network hardware pricing from FS.com.

Components	512-node Fat-Tree			2-OCS			4-OCS			2-Robot R2D2			4-Robot R2D2		
	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty	Cost (\$)	Power (W)	Qty
N8560-32C 32 x 100Gb switch	68.7K	450	48	—	—	—	—	—	—	—	—	—	—	—	—
OCS Unit (576 Ports)	—	—	—	~300K	~200	2	~300K	~200	4	—	—	—	—	—	—
R2D2 Unit (576 Ports)	—	—	—	—	—	—	—	—	—	5.4K	120	2	5.4K	120	4
Mellanox ConnectX-5 100Gb NIC	429	19.3	512	429	19.3	512	429	19.3	512	429	19.3	512	429	19.3	512
Copper 100Gb Cable (2m)	54	Incl. in NIC	512	—	—	—	—	—	—	—	—	—	—	—	—
Optical Transceivers	74	Incl. in NIC	1024	—	—	—	—	—	—	—	—	—	—	—	—
Optical 100G to 2x50G Transceivers	—	—	—	271	Incl. in NIC	512	—	—	—	271	Incl. in NIC	512	—	—	—
Optical 100G to 4x25G Transceivers	—	—	—	—	—	—	271	Incl. in NIC	512	—	—	—	271	Incl. in NIC	512
Optical Cables (3/5/10m)	10	Incl. in NIC	512	—	—	—	—	—	—	—	—	—	—	—	—
Optical Cables (10m)	—	—	—	Incl'd	Incl. in NIC	1024	Incl'd	Incl. in NIC	2048	Incl'd	Incl. in NIC	1024	Incl'd	Incl. in NIC	2048
Total	658K	31.5K		957K	10.3K		1.57M	10.6K		369K	10.1K		3.80M	10.4K	

TABLE VII: Comparison of cost and power components for a 100G 512-node fat-tree, a 2-OCS network, a 4-OCS network, a 2-robot R2D2 fabric, and a 4-robot R2D2 fabric. OCS pricing and power scaled from [38], [70]. Network hardware pricing from FS.com.